

92. Un tableau de struct, parcouru par une boucle

Quatre ennemis, un tableau, une boucle : le code d'un seul, appliqué à tous.

1. Ce qu'il faut comprendre

`Ennemi troupe[4];` : un **tableau** dont chaque case est une **struct**. On écrit `troupe[i].x` — **d'abord** la case, **puis** le champ.

Le code d'un ennemi est écrit **une** fois, dans une boucle `for` sur `i`. Qu'il y ait 4 ennemis ou 40, le code est le même : on change `COMBIEN`, c'est tout. C'est là que tout le cours se rejoint — variables, conditions, boucles, tableaux et struct dans le même bloc.

Chaque ennemi a son **sens** : 1 vers la droite, 0 vers la gauche. Quand il touche un bord, on **retourne** son sens. Deux `if` pour avancer, deux pour faire demi-tour.

Les ennemis ne bougent qu'**une image sur huit** — une seule condition autour de toute la boucle suffit. Et chacun est initialisé par une **boucle aussi** : sa ligne et sa colonne de départ se **calculent** à partir de `i`.

À toi : donne à chaque ennemi sa propre vitesse — un champ `lenteur`, et `images() % troupe[i].lenteur`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **645 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         449 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
AdresseCase    10 o ← texte() - trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles - 1 lettre sur 43
" "            1 o ← texte() - un texte gravé
"x"            1 o ← texte() - un texte gravé
```

Lettres gravées : X

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte()  × 2   120 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
image()  × 1    3 o sur place, dans main() + AttendreImage (28 o)
images() × 1    2 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : quatre ennemis X font des allers-retours sur l'écran.
//
// Ce qui est nouveau : un TABLEAU de struct.
//  Ennemi truppe[4]   = 4 ennemis ; chacun a son x, son y et son sens.
//  truppe[i].x        = la case x de l'ennemi numéro i.
//  Une seule boucle for s'occupe de tous les ennemis.

#include <texte>    // écrit un texte à l'écran

```

2. UNE CONSTANTE — COMBIEN

```

const uint8_t COMBIEN = 4;    // Le nombre d'ennemis. const : il ne change jamais.

```

3. UN ENREGISTREMENT — ENNEMI

```

struct Ennemi {                // Le modèle d'un ennemi :
    uint8_t x;                 // sa colonne,
    uint8_t y;                 // sa ligne,
    uint8_t sens;             // 1 : vers la droite, 0 : vers la gauche
};

```

4. UN TABLEAU — TROUPE

```

Ennemi truppe[COMBIEN];       // 4 ennemis : truppe[0] à truppe[3].

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {                    // Le jeu commence ici.
    for (uint8_t i = 0; i < COMBIEN; i++) { // Pour chaque ennemi i (0 à 3) :
        truppe[i].x = 2 + i * 4;    // colonne 2, 6, 10, 14
        truppe[i].y = 4 + i * 3;    // ligne 4, 7, 10, 13
        truppe[i].sens = i % 2;    // reste par 2 : 0, 1, 0, 1 (un sur deux part à droite)
    }

    while (true) {              // La boucle du jeu :
        image();
        if (images() % 8 != 0) continue; // une image sur huit (sinon, on saute la suite)

        for (uint8_t i = 0; i < COMBIEN; i++) { // Pour chaque ennemi :
            texte(truppe[i].x, truppe[i].y, " "); // efface-le à sa place actuelle,

            if (truppe[i].sens == 1) truppe[i].x++; // avance dans son sens,
            else truppe[i].x--;

            if (truppe[i].x == 19) truppe[i].sens = 0; // au bord droit : repart à gauche,
            if (truppe[i].x == 0) truppe[i].sens = 1; // au bord gauche : repart à droite,

            texte(truppe[i].x, truppe[i].y, "X"); // dessine-le à sa nouvelle place.
        }
    }
}

```

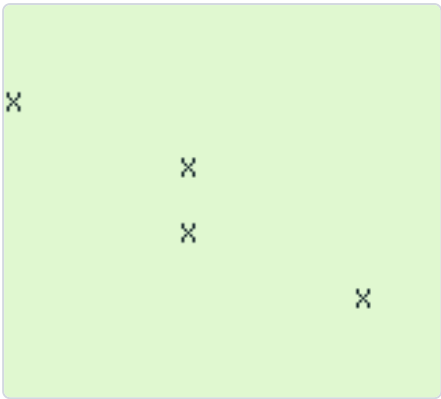
3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est

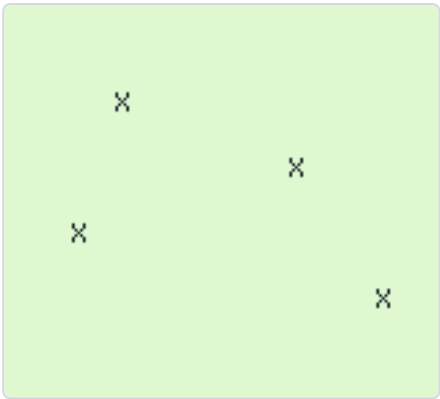
la vérité sur ce programme.



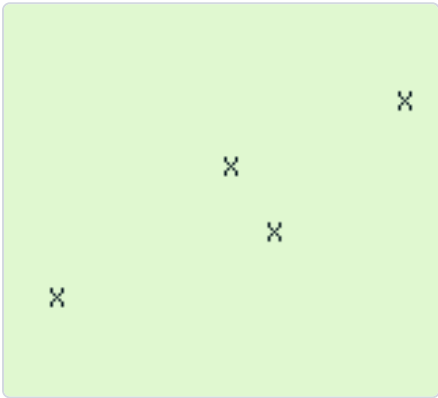
la toute première image — la console vient de démarrer — image 1



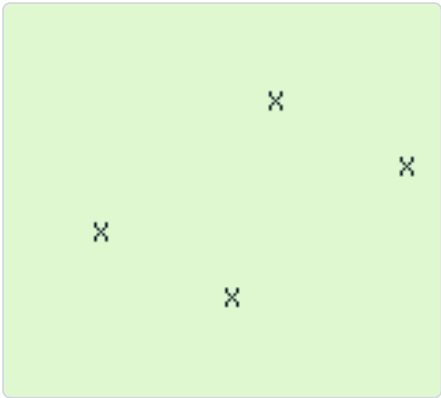
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Quatre X sur quatre lignes, qui vont et viennent d'un bord à l'autre, chacun dans son sens.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ jamais deux X sur une ligne : l'ancienne case est effacée
- ✓ chacun a parcouru plusieurs cases (20, 20, 20, 20 cases)
- ✓ chacun a touché un bord, et a fait demi-tour

Ensuite : **93. Une méthode : la fonction qui connaît son objet** — chapitre 14 : Les struct : ce qui va ensemble. · gameboy3 — écrire une cartouche Game Boy en C++